Material do Estudante — Grafos, BFS e DFS

Disciplina: Estrutura de Dados II — Ciência da Computação

Estudo de caso: pane relacionada ao GPS e impactos em voos no Aeroporto de Brasília (G1, 22 set. 2026).

Referência principal: GOMES, Paulo César Rodacki. *Grafos: conceitos fundamentais, algoritmos e aplicações*. Blumenau: Editora IFC, 2022.

> Importante: esta atividade é uma simulação computacional, não um modelo de navegação aeronáutica operacional. A notícia motiva o problema; os bloqueios e rotas do exercício são fictícios.

1. Situação-problema e objetivo

Uma indisponibilidade de recursos de navegação pode provocar replanejamento operacional. Neste laboratório, representaremos conexões hipotéticas entre cidades como um grafo e procuraremos caminhos entre Brasília e Curitiba, comparando os algoritmos BFS e DFS. O objetivo não é indicar rotas de voo reais, mas estudar conectividade e algoritmos de busca.

Sua missão: projetar um grafo próprio com pelo menos sete vértices, implementar BFS e DFS em Python, simular a remoção de uma conexão e justificar a solução encontrada.

2. Fundamentos de grafos — Gomes (2022), capítulo 1

Um grafo é uma estrutura matemática representada por G = (V, E): V é o conjunto de vértices e E é o conjunto de arestas. No estudo de caso, vértices podem representar cidades ou pontos de conexão fictícios e arestas representam ligações permitidas no modelo. Dois vértices são adjacentes quando estão ligados por uma aresta. Um caminho é uma sequência de vértices conectados por arestas.

Um grafo não dirigido permite interpretar uma ligação nos dois sentidos; um grafo dirigido usa arcos orientados e distingue origem e destino. A rede de voos real é muito mais complexa e exige regras operacionais que não são modeladas aqui. Diferentemente de uma árvore binária, um grafo pode apresentar ciclos e múltiplos caminhos entre os mesmos vértices.

3. Representação computacional — Gomes (2022), capítulo 2

A lista de adjacência registra, para cada vértice, seus vizinhos. Em Python, um dicionário cujos valores são listas implementa essa representação de forma clara. No caso de um grafo não dirigido, cada conexão deve aparecer nas listas dos dois extremos.

```
grafo = {
    "Brasília": ["Goiânia", "Belo Horizonte"],
    "Goiânia": ["Brasília", "São Paulo", "Cuiabá"],
    "Belo Horizonte": ["Brasília", "Rio de Janeiro"],
    "São Paulo": ["Goiânia", "Curitiba", "Rio de Janeiro"],
    "Cuiabá": ["Goiânia", "Curitiba"],
    "Rio de Janeiro": ["Belo Horizonte", "São Paulo", "Curitiba"],
    "Curitiba": ["São Paulo", "Cuiabá", "Rio de Janeiro"]
}
```

Interprete: grafo["Brasília"] apresenta os vizinhos imediatos de Brasília; não representa uma lista de voos comerciais efetivamente disponíveis.

4. Busca em largura — BFS (capítulo 3, seção 3.2)

BFS (*Breadth-First Search*) explora primeiro os vértices mais próximos da origem em número de arestas. Utiliza uma fila FIFO: o primeiro elemento inserido é o primeiro removido. Em um grafo não ponderado, BFS encontra um caminho com o menor número de arestas, se houver caminho. Não garante menor tempo, distância ou consumo de combustível.

Passo a passo: (1) inserir a origem na fila; (2) marcar a origem como visitada; (3) retirar o primeiro vértice; (4) verificar se é o destino; (5) enfileirar vizinhos ainda não visitados; (6) repetir até encontrar o destino ou esvaziar a fila.

5. Busca em profundidade — DFS (capítulo 3, seção 3.3)

DFS (*Depth-First Search*) aprofunda um ramo antes de retornar e explorar outros. Pode usar pilha LIFO ou recursão. DFS identifica um caminho, caso exista, mas não garante a menor quantidade de arestas. A ordem de vizinhos influencia o caminho encontrado. O controle de visitados evita repetição infinita em grafos com ciclos.

Passo a passo: (1) colocar a origem na pilha; (2) retirar o último elemento; (3) verificar o destino; (4) explorar os vizinhos ainda não visitados; (5) continuar até encontrar o destino ou esgotar a pilha.

6. Implementação guiada em Python

Copie e execute o código a seguir no Google Colab ou em um arquivo .py. Compare os caminhos retornados pelos algoritmos.

```
from collections import deque

grafo = {
    "Brasília": ["Goiânia", "Belo Horizonte"],
    "Goiânia": ["Brasília", "São Paulo", "Cuiabá"],
    "Belo Horizonte": ["Brasília", "Rio de Janeiro"],
    "São Paulo": ["Goiânia", "Curitiba", "Rio de Janeiro"],
    "Cuiabá": ["Goiânia", "Curitiba"],
    "Rio de Janeiro": ["Belo Horizonte", "São Paulo", "Curitiba"],
    "Curitiba": ["São Paulo", "Cuiabá", "Rio de Janeiro"]
}

def bfs(rede, origem, destino):
    fila = deque([(origem, [origem])])
    visitados = {origem}
    while fila:
        atual, caminho = fila.popleft()
        if atual == destino:
            return caminho
        for vizinho in rede.get(atual, []):
            if vizinho not in visitados:
                visitados.add(vizinho)
                fila.append((vizinho, caminho + [vizinho]))
    return None

def dfs(rede, origem, destino):
    pilha = [(origem, [origem])]
    visitados = set()
    while pilha:
        atual, caminho = pilha.pop()
        if atual in visitados:
            continue
        visitados.add(atual)
        if atual == destino:
            return caminho
        for vizinho in reversed(rede.get(atual, [])):
            if vizinho not in visitados:
                pilha.append((vizinho, caminho + [vizinho]))
    return None

def bloquear_conexao(rede, a, b):
    copia = {no: vizinhos.copy() for no, vizinhos in rede.items()}

    if b in copia.get(a, []):
        copia[a].remove(b)
    if a in copia.get(b, []):
        copia[b].remove(a)
    return copia

origem, destino = "Brasília", "Curitiba"
for nome, rede in [("Normal", grafo),
                   ("Bloqueio Goiânia–São Paulo", bloquear_conexao(grafo, "Goiânia", "São Paulo"))]:
    print("\nCenário:", nome)
    print("BFS:", bfs(rede, origem, destino))
    print("DFS:", dfs(rede, origem, destino))
```

Entenda as partes do código

- deque: implementa fila eficiente para BFS; popleft() remove o primeiro elemento.

- pilha.pop(): remove o último elemento inserido e produz o comportamento LIFO do DFS.

- visitados: evita revisitar vértices em uma rede com ciclos.

- caminho + [vizinho]: mantém a sequência de vértices até cada candidato.

- bloquear_conexao: cria uma cópia do grafo e remove a aresta nos dois sentidos.

- None: significa que o algoritmo não encontrou um caminho no modelo fornecido.

7. Experimentos e interpretação

Experimento A: execute BFS e DFS na rede completa. Registre a sequência de cidades e conte as arestas de cada caminho.

Experimento B: remova a conexão Goiânia–São Paulo. Execute novamente e compare. O bloqueio é uma hipótese didática, não uma consequência comprovada da pane relatada.

Experimento C: remova uma segunda conexão e verifique se ainda existe caminho. Justifique o resultado usando conectividade.

Experimento D: altere a ordem de vizinhos em uma lista. Explique por que o resultado de DFS pode mudar sem que o grafo mude.

8. Desafio individual: desenvolva sua solução

1. Formule uma pergunta de pesquisa computacional relacionada à conectividade da rede simulada.

2. Desenhe um grafo com pelo menos sete vértices e conexões justificadas.

3. Implemente a lista de adjacência e BFS e DFS em Python.

4. Permita que o usuário informe origem e destino, validando nomes inexistentes.

5. Simule o bloqueio de pelo menos uma aresta, sem alterar permanentemente a rede original.

6. Mostre os caminhos retornados e a quantidade de arestas. Trate corretamente casos sem caminho.

7. Compare BFS e DFS e explique as limitações da solução.

8. Apresente sua implementação funcionando, com os testes executados.

Entregáveis

- solucao_grafos.py ou notebook .ipynb executável.

- README.md com problema, objetivo, instruções, estrutura do grafo e resultados.

- Diagrama do grafo e evidências dos testes.

- Apresentação individual sugerida de 5 minutos, demonstrando a execução e justificando escolhas.

Critérios sugeridos de avaliação (10 pontos)

• Critério • Pontos •

• --- • ---: •

• Modelagem do problema e grafo • 2,0 •

• Implementação correta de BFS e DFS • 3,0 •

• Simulação e testes de contingência • 2,0 •

• Comparação crítica e limitações • 1,5 •

• Organização e apresentação da solução • 1,5 •

9. Questões para reflexão

1. Por que um grafo representa melhor a rede de conexões do que uma árvore binária?

2. Em qual condição BFS encontra o menor caminho? O que significa “menor”?

3. Por que DFS pode retornar um caminho diferente de BFS?

4. O que ocorre se não utilizarmos um conjunto de visitados?

5. Quais informações adicionais seriam necessárias para avaliar tempo ou custo de uma rota?

6. Por que o resultado do algoritmo não pode ser usado para orientar um voo real?

10. Referências

GOMES, Paulo César Rodacki. Grafos: conceitos fundamentais, algoritmos e aplicações. Blumenau: Editora IFC, 2022. ISBN 978-65-88089-12-5. (Capítulos 1, 2 e 3; capítulo 7 para aprofundamento.)

G1 DISTRITO FEDERAL. Pane em GPS afeta voos da Azul no Aeroporto de Brasília; passageiros reclamam de atrasos. 22 set. 2026. Disponível em: https://g1.globo.com/df/distrito-federal/noticia/2026/09/22/pane-em-gps-afeta-voos-da-azul-no-aeroporto-de-brasilia-passageiros-reclamam-de-atrasos.ghtml.